

Lecture Notes

# **Numerical Optimization**

Winter Term 2025/2026

Michael C. Lehn

Institute of Numerical Mathematics  
Ulm University

# Contents

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                             | <b>4</b>  |
| <b>2</b> | <b>Nonlinear Least Squares Problems</b>         | <b>5</b>  |
| §1       | Problem Statement . . . . .                     | 5         |
| §2       | Test Problems . . . . .                         | 10        |
| §3       | Derivatives of the Objective Function . . . . . | 14        |
| §4       | Newton–Raphson Method . . . . .                 | 17        |
| §5       | Gauss–Newton . . . . .                          | 20        |
| §6       | Outlook: Levenberg–Marquardt method . . . . .   | 23        |
| <b>3</b> | <b>Unconstrained optimization</b>               | <b>24</b> |
| §1       | Optimality conditions and convexity . . . . .   | 25        |
| §2       | Direct search methods . . . . .                 | 28        |
| §3       | Gradient descent methods . . . . .              | 28        |
| §4       | Trust-Region Methods . . . . .                  | 28        |
| <b>A</b> | <b>Supplementary Notes to Chapters</b>          | <b>29</b> |
| §1       | Least Square . . . . .                          | 29        |

## List of Algorithms

|   |                                                  |    |
|---|--------------------------------------------------|----|
| 1 | <a href="#">Gauss–Newton algorithm</a> . . . . . | 22 |
|---|--------------------------------------------------|----|

## Listings

|     |                                                                                  |    |
|-----|----------------------------------------------------------------------------------|----|
| 2.1 | Function <code>create_circ_data</code> : Generate synthetic circle data. . . . . | 13 |
| 2.2 | Function <code>draw_circle</code> : Visualize data and fitted circles. . . . .   | 13 |

# Chapter 1 Introduction

The problems considered in this chapter arise in practical applications where measurements are affected by noise and models are nonlinear. In general, such problems are not solvable in a strict mathematical sense. However, they do not arise arbitrarily but have a specific structure and origin that can and must be exploited.

Numerical mathematics provides the framework to deal with such situations. By using approximation, iteration, and careful analysis, it becomes possible to obtain practically useful results even when an exact solution cannot be found.

This is not a compromise of mathematics—it is its frontier. Numerical mathematics is, in this sense, the spearhead of mathematics.

## Chapter 2 Nonlinear Least Squares Problems

In the course on Numerical Linear Algebra, we studied the *linear* least squares problem, that is, minimization problems of the form

$$\|Ax - b\|_2^2 \rightarrow \min$$

for a full-rank matrix  $A \in \mathbb{R}^{m \times n}$  and a vector  $b \in \mathbb{R}^m$  with  $m \geq n$ . Defining

$$f : \mathbb{R}^n \rightarrow \mathbb{R}, \quad f(x) = \|Ax - b\|_2^2,$$

we saw that

$$f(\hat{x}) = \min_{x \in \mathbb{R}^n} f(x) \iff \nabla f(\hat{x}) = \mathbf{0}.$$

Because  $f$  is convex in  $x$ , this necessary condition is also sufficient for a global minimum. Since

$$\nabla f(x) = 2A^T(Ax - b),$$

the minimizer  $\hat{x}$  satisfies the so-called *normal equations*

$$A^T A x = A^T b.$$

In the Numerical Linear Algebra course, the emphasis was on algorithms suitable for solving these equations efficiently and stably. For example, the Cholesky decomposition exploits the fact that  $A^T A$  is symmetric and positive definite, while the QR decomposition allows one to compute the solution without explicitly forming  $A^T A$ , which improves numerical stability.

In the following, we will generalize the problem by allowing the objective function  $f$  to be nonlinear in  $x$ . This is highly relevant in practice, since many interesting problems fall into this category. From a theoretical perspective, however, nonlinear least squares problems are considerably more challenging.

### §1 Problem Statement

In this chapter, we consider objective functions

$$f : \mathbb{R}^n \rightarrow \mathbb{R}$$

of the special form

$$f(x) = \frac{1}{2} \sum_{i=1}^m F_i(x)^2,$$

where the functions  $F_i : \mathbb{R}^n \rightarrow \mathbb{R}$ ,  $1 \leq i \leq m$ , are assumed to be twice continuously differentiable. Equivalently, we can write

$$f(x) = \frac{1}{2} F(x)^T F(x) = \frac{1}{2} \|F(x)\|_2^2, \quad F(x) = \begin{pmatrix} F_1(x) \\ \vdots \\ F_m(x) \end{pmatrix}.$$

To facilitate later reference, we summarize this specific class of problems in the following definition.

**Definition 2.1.1** (Least Squares Problem)

Let  $F_1, \dots, F_m \in C^2(\mathbb{R}^n; \mathbb{R})$  with  $m, n \geq 1$ , and define

$$f : \mathbb{R}^n \rightarrow \mathbb{R}, \quad f(x) = \frac{1}{2} \sum_{i=1}^m F_i(x)^2.$$

A point  $\hat{x} \in \mathbb{R}^n$  is called a (global) solution of the *least squares problem* if

$$f(\hat{x}) = \min_{x \in \mathbb{R}^n} f(x).$$

□

Having defined the general problem, we now verify that it indeed represents a genuine generalization of the least squares problem encountered in the course on Numerical Linear Algebra.

**Example 2.1.2** (Linear least squares as a special case)

Let  $a_1, \dots, a_m \in \mathbb{R}^n$  and  $b_1, \dots, b_m \in \mathbb{R}$ , and define

$$F_i(x) = a_i^T x - b_i, \quad 1 \leq i \leq m.$$

Then

$$\begin{aligned} f(x) &= \frac{1}{2} \sum_{i=1}^m (a_i^T x - b_i)^2 \\ &= \frac{1}{2} \|Ax - b\|_2^2, \end{aligned}$$

where

$$A = \begin{pmatrix} a_1^T \\ \vdots \\ a_m^T \end{pmatrix}, \quad b = \begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix}.$$

Clearly,

$$f(\hat{x}) = \min_{x \in \mathbb{R}^n} f(x) \iff \|A\hat{x} - b\|_2^2 = \min_{x \in \mathbb{R}^n} \|Ax - b\|_2^2.$$

Thus, this special case is equivalent to the familiar *linear least squares problem*. If  $m \geq n$  and  $A$  has full column rank, the minimizer is unique.

Hence, the nonlinear least squares problem truly generalizes the classical linear one: the objective function differs only by the factor  $\frac{1}{2}$ , which is introduced merely for convenience, as it simplifies the expression for the necessary condition of optimality (see (2.1) below). □

Despite the rather strong assumption that the objective function  $f$  is twice continuously differentiable and, by construction, bounded below by 0, no general statement can be made about the existence (or even uniqueness) of a solution.

**Example 2.1.3**

Let  $m = n = 1$ .

(a) For  $F(x) = F_1(x) = 1 - x^2$  we have

$$f(x) = \frac{1}{2}F(x)^2 = \frac{1}{2}(1 - x^2)^2.$$

Clearly,  $f(x) = 0$  if and only if  $x = \pm 1$ . Hence, a solution exists, but it is not unique.

(b) For  $F(x) = F_1(x) = \frac{1}{1+x^2}$  we obtain

$$f(x) = \frac{1}{2(1+x^2)^2} > 0 \quad \text{for all } x \in \mathbb{R}.$$

Since

$$\lim_{x \rightarrow \infty} f(x) = 0,$$

no global minimizer exists.

□

If a solution  $\hat{x}$  exists, that is, if the global minimum of  $f$  is attained, then  $\hat{x}$  is in particular also a local minimum. A necessary condition for a local minimum (or, more generally, for a stationary point) is that the gradient vanishes:

$$f'(\hat{x}) = \nabla f(\hat{x})^T = \mathbf{0}^T. \quad (2.1)$$

A sufficient condition for a local minimum is that

$$\begin{aligned} \nabla f(\hat{x}) &= \mathbf{0} \quad \text{and} \quad r^T H_f(\hat{x}) r > 0 \quad \forall r \neq \mathbf{0} \\ \Rightarrow \exists \varepsilon > 0 : f(x) &> f(\hat{x}) \quad \forall x : \|x - \hat{x}\|_2 < \varepsilon. \end{aligned} \quad (2.2)$$

For completeness, these well-known results from analysis are reviewed in Appendix §1. That appendix also summarizes the notation and conventions used throughout this text, such as  $\nabla f$  for the gradient (the  $n \times 1$  Jacobian matrix) and  $H_f$  for the Hessian matrix.

However, the sufficient condition for a local minimum given in (2.2) is not always conclusive, as the following example shows.

**Example 2.1.4** (Stationary points and higher derivatives)

Let  $m = n = 1$ .

(a) For  $F_1(x) = x^2$  we have

$$f(x) = \frac{1}{2}F_1(x)^2 = \frac{1}{2}x^4,$$

and  $\hat{x} = 0$  is clearly the point of the global minimum. Here

$$f'(0) = f''(0) = f^{(3)}(0) = 0, \quad f^{(4)}(0) = 4! > 0.$$



In general, if  $f$  is  $2k$ -times continuously differentiable and

$$f'(\hat{x}) = f''(\hat{x}) = \cdots = f^{(2k-1)}(\hat{x}) = 0, \quad f^{(2k)}(\hat{x}) > 0,$$

then  $\hat{x}$  is a local minimizer.

For  $n > 1$ ,  $f''(\hat{x})$  is a matrix (the Hessian). Checking its definiteness is numerically feasible, as known from the course on Numerical Linear Algebra. We also know the computational cost associated with this task. Higher derivatives (3rd, 4th, etc.) would be tensors. In principle, similar sufficient conditions could be formulated using them, but the computational effort grows rapidly. Since we only assume  $f \in C^2(\mathbb{R}^n)$ , we will not pursue this idea further, as higher derivatives may not even exist.

- (b) Even if we allow  $f$  to be infinitely often differentiable, examining higher derivatives alone may not suffice. Consider the classical example

$$F_1(x) = \begin{cases} e^{-1/x^2}, & x \neq 0, \\ 0, & x = 0. \end{cases}$$

Then  $F_1$  is infinitely differentiable, and  $x = 0$  is a global minimum of  $F_1$ , but

$$F_1^{(k)}(0) = 0 \quad \text{for all } k \in \mathbb{N}.$$

The same holds for

$$f(x) = \frac{1}{2}F_1(x)^2,$$

where all derivatives vanish at  $x = 0$ , even though  $x = 0$  is a strict global minimizer. Thus, higher-order derivatives alone do not necessarily provide sufficient information to identify local minima.

□

For a numerical method to be considered feasible, it is therefore reasonable to require merely that it produces stationary points of the objective function. Whether a stationary point corresponds to a local minimum can then be examined separately.

However, even this task is far from trivial. A conceivable strategy for finding the solution would be to determine all local minima and then select the one with the smallest function value. For such an approach to be theoretically feasible, it must at least be guaranteed that there exist only finitely many local minima.

To illustrate what “not feasible” may look like in practice and to appreciate the inherent difficulty of the problem, consider the following example.

**Example 2.1.5** (Infinitely many non-global local minima)

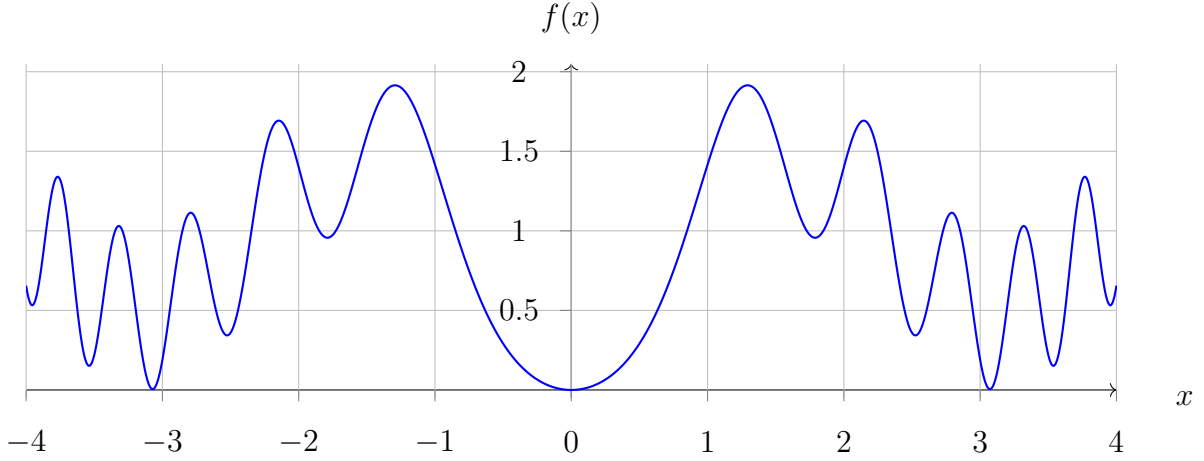
Let

$$f(x) = \sin^2(x) + \sin^2(x^2).$$

Then  $f(0) = 0$  and  $f(x) > 0$  for all  $x \neq 0$ , so  $\hat{x} = 0$  is the unique global minimizer. Nevertheless,  $f$  possesses infinitely many local minima. This can already be seen intuitively from the graph of  $f$  (see Figure 2.1), whose oscillations become increasingly dense for large values of  $|x|$ .

A proof of this statement is given at the end of this section in Remark 2.1.6. Before that, we discuss what such examples imply for numerical treatment.

□



**Figure 2.1:** Graph of  $f(x) = \sin^2(x) + \sin^2(x^2)$  on  $[-4, 4]$ .

In situations like this, a numerical optimization method can at best be expected to generate a sequence of local minimizers converging towards the global one. In general, however, the local minimizers cannot be determined analytically but only numerically. Hence, even finding a single local minimizer requires a numerical method that, under suitable assumptions, produces a sequence converging to a local stationary point. Whether such a stationary point corresponds to a local minimum, a saddle point, or a local maximum can, in general, only be determined *a posteriori*.

One should therefore not expect numerical optimization to provide a magical black box capable of solving such problems automatically. After this somewhat sobering realization, it is worth emphasizing that there are still good reasons to be optimistic: objective functions arising in applications do not fall from the sky. In practice, we usually know much more about them than merely that they are twice continuously differentiable. What we can develop is therefore not a single universal algorithm that solves all problems, but rather a toolbox of methods that—under appropriate assumptions—can handle specific subclasses of problems efficiently. Using this toolbox effectively requires understanding the underlying methods and their limitations, in particular the conditions under which they are guaranteed to work.

Before we turn to an application example in the next section, we give here the proof promised in Example 2.1.5.

**Remark 2.1.6 (Proof of Example 2.1.5)**

We consider

$$f(x) = \sin^2 x + \sin^2 x^2,$$

with

$$\begin{aligned} f'(x) &= 2 \sin x \cdot \cos x + 2 \sin x^2 \cdot \cos x^2 \cdot 2x \\ &= \sin 2x + 2x \sin 2x^2. \end{aligned}$$

We show the following three statements:

- (a)  $x = 0$  is a global minimizer.

Since both terms in  $f(x)$  are nonnegative and vanish simultaneously only for  $x = 0$ ,

we have

$$f(x) \geq 0 \quad \text{for all } x \in \mathbb{R}, \quad f(0) = 0.$$

Hence  $\hat{x} = 0$  is a global minimizer with  $f(\hat{x}) = 0$ .

(b)  $x = 0$  is the unique global minimizer.

The equality  $f(x) = 0$  requires  $\sin x = 0$  and  $\sin x^2 = 0$  simultaneously. This means

$$x = k\pi \quad \text{and} \quad x^2 = n\pi \quad \text{for some } k, n \in \mathbb{Z}.$$

Combining both gives  $k^2\pi^2 = n\pi$ , i.e.  $n = k^2\pi$ . As  $\pi$  is irrational, this is impossible for integer  $n, k$  unless  $k = n = 0$ . Therefore  $x = 0$  is the only point where  $f(x) = 0$ , and thus the unique global minimizer.

(c)  $f$  has infinitely many local minima.

Define

$$a_n = \sqrt{\frac{\pi}{4}(4n+1)}, \quad b_n = \sqrt{\frac{\pi}{4}(4n+3)}, \quad n \in \mathbb{N}.$$

Then

$$\sin 2a_n^2 = 1, \quad \sin 2b_n^2 = -1.$$

Because

$$b_n > a_n \geq \sqrt{\frac{\pi}{4}} > \frac{1}{2},$$

we have

$$\begin{aligned} f'(a_n) &= \sin 2a_n + 2a_n > \sin 2a_n + 1 \geq 0, \\ f'(b_n) &= \sin 2b_n - 2b_n < \sin 2b_n - 1 \leq 0. \end{aligned}$$

And since  $a_n < b_n < a_{n+1}$ , the intermediate value theorem ensures at least one zero of  $f'$  in each of the intervals  $(a_n, b_n)$  and  $(b_n, a_{n+1})$ . Hence  $f'$  changes sign infinitely often, and  $f$  possesses infinitely many local minima and maxima.

In particular,  $x = 0$  is the unique global minimizer, while all other local minima satisfy  $f(x) > 0$ . □

## §2 Test Problems

In the following, three related problems are introduced that will later serve as test cases for numerical methods designed to solve least squares problems. Each problem is constructed such that the first and second derivatives of the objective function can be computed analytically — which, of course, is not generally possible in practical applications.

In all three cases, we are given points  $(x_i, y_i)^T \in \mathbb{R}^2$  for  $1 \leq i \leq m$ , which are assumed to lie approximately on the boundary of a circle with true center  $(\bar{c}_x, \bar{c}_y)^T$  and true radius  $\bar{r}$ , according to an underlying model, but are affected by stochastic noise.

Formally, each observed data point can be expressed as

$$\begin{aligned} x_i &= \bar{x}_i + \varepsilon_{x,i}, \\ y_i &= \bar{y}_i + \varepsilon_{y,i}, \end{aligned} \quad 1 \leq i \leq m,$$

where the ideal (noise-free) points  $(\bar{x}_i, \bar{y}_i)$  are located on the circumference of a circle with true parameters  $(\bar{c}_x, \bar{c}_y, \bar{r})$  and satisfy

$$\begin{aligned}\bar{x}_i &= \bar{c}_x + \bar{r} \cos \varphi_i, \\ \bar{y}_i &= \bar{c}_y + \bar{r} \sin \varphi_i,\end{aligned}\quad 1 \leq i \leq m.$$

The angles  $\varphi_i$  are independently and uniformly distributed as

$$\varphi_i \stackrel{\text{iid}}{\sim} \mathcal{U}(0, 2\pi),$$

and the noise terms are independent and identically distributed as

$$\varepsilon_{x,i}, \varepsilon_{y,i} \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2).$$

For a single data point  $(x_i, y_i)^T$  we define the residual

$$F_i(r, c_x, c_y) = \sqrt{(x_i - c_x)^2 + (y_i - c_y)^2} - r, \quad (2.3)$$

and the least-squares objective

$$f(r, c_x, c_y) = \frac{1}{2} \sum_{i=1}^m F_i(r, c_x, c_y)^2. \quad (2.4)$$

If the data are noise-free (formally,  $\sigma = 0$ ), we have

$$f(\bar{r}, \bar{c}_x, \bar{c}_y) = 0.$$

For noisy data with  $\sigma > 0$ , this equality no longer holds exactly. The expected value of  $f(\bar{r}, \bar{c}_x, \bar{c}_y)$  is strictly positive due to the bias introduced by the noise. Nevertheless, for small noise levels, the global minimizer of  $f$  will typically be close to the true parameters  $\bar{r}, \bar{c}_x$ , and  $\bar{c}_y$ , and can therefore serve as a useful estimator of the unknown true parameters.

We now consider the following three test problems, with objectives  $f_{\text{ex1}}, f_{\text{ex2}}, f_{\text{ex3}}$ .

- **Example 1 (known center at the origin)**

The center of the circle is known to be at  $(c_x, c_y) = (0, 0)$ , while the radius  $r$  is unknown. Although negative values of  $r$  have no physical meaning, we deliberately allow  $r < 0$  so that the problem remains unconstrained. This leads to

$$f_{\text{ex1}}: \mathbb{R} \rightarrow \mathbb{R}, \quad f_{\text{ex1}}(r) := f(r, 0, 0). \quad (2.5)$$

A particular advantage of this setting is that both  $f$  and its derivative  $f'$  can be easily visualized.

- **Example 2 (partially known center)**

The circle center is partially known:  $c_y = 0$  is fixed, whereas  $c_x$  and  $r$  are unknown. This leads to

$$f_{\text{ex2}}: \mathbb{R}^2 \rightarrow \mathbb{R}, \quad f_{\text{ex2}}(r, c_x) := f(r, c_x, 0). \quad (2.6)$$

In this case, a contour plot of  $f(c_x, r)$  can be generated, giving intuitive insight into the structure of the objective function and the distribution of its extrema.

- **Example 3 (unknown center and radius)**

The most general case, where  $r$ ,  $c_x$ , and  $c_y$  are all unknown. This leads to

$$f_{\text{ex3}}: \mathbb{R}^3 \rightarrow \mathbb{R}, \quad f_{\text{ex3}}(r, c_x, c_y) := f(r, c_x, c_y). \quad (2.7)$$

Here no parameters are fixed, making the problem fully nonlinear and three-dimensional.

[Listing 2.1](#) shows the implementation of the function

```
function [x, y] = create_circ_data(m, r_, cx_, cy_, sigma)
```

which generates synthetic test data for the problems described above. Its parameters have the following meaning:

- **m** – number of data points,
- **r\_** – true radius, corresponding to  $\bar{r}$ ,
- **cx\_**, **cy\_** – true circle center, corresponding to  $\bar{c}_x$  and  $\bar{c}_y$ ,
- **sigma** – standard deviation of the normally distributed noise.

The function returns two column vectors

$$x = (x_1, \dots, x_m)^T, \quad y = (y_1, \dots, y_m)^T,$$

representing the observed data points.

To build intuition for the three test problems, it is instructive to visualize the noisy data together with the “true” and “estimated” circles. [Listing 2.2](#) shows the implementation of the function

```
function draw_circle(x, y, r_, cx_, cy_, r, cx, cy)
```

Its arguments are:

- **x**, **y** – data vectors as returned by `create_circ_data`,
- **r\_**, **cx\_**, **cy\_** – true circle parameters  $(\bar{r}, \bar{c}_x, \bar{c}_y)$ ,
- **r**, **cx**, **cy** – estimated circle parameters.

The function plots the data points  $(x_i, y_i)$ , the true circle (in blue, with radius  $\bar{r}$  and center  $(\bar{c}_x, \bar{c}_y)$ ), and the estimated circle (in red, with radius  $r$  and center  $(c_x, c_y)$ ). This visualization provides an immediate geometric impression of how well the estimated parameters fit the noisy data.

[Figure 2.2](#) illustrates characteristic aspects of the test problems introduced above. The three subfigures do not visualize all cases ( $f_{\text{ex3}}$  would be three-dimensional and is therefore difficult to display), but they provide geometric intuition for the first two examples and for the general nature of the least squares objectives.

1. [Subfigure 2.2a](#) shows how the noisy data points are scattered around the “true” circle. The data were generated with `[x, y] = create_circ_data(500, 10, 0, 0, 1)` and visualized using `draw_circle(x, y, 10, 0, 0, 10, 0, 0)`.

**Listing 2.1:** *Function create\_circ\_data: Generate synthetic circle data.*

```

1 function [x, y] = create_circ_data(m, r_, cx_, cy_, sigma)
2
3     arguments
4         m (1,1) double {mustBeInteger, mustBeNonnegative}
5         r_ (1,1) double {mustBeFinite}
6         cx_ (1,1) double {mustBeFinite}
7         cy_ (1,1) double {mustBeFinite}
8         sigma (1,1) double {mustBeNonnegative} = 1
9     end
10
11     if m == 0
12         x = zeros(0,1);
13         y = zeros(0,1);
14         return
15     end
16
17     phi = 2 * pi * rand(m,1);
18     noise = sigma * randn(m,2);
19
20     x = cx_ + r_ * cos(phi) + noise(:, 1);
21     y = cy_ + r_ * sin(phi) + noise(:, 2);
22 end

```

**Listing 2.2:** *Function draw\_circle: Visualize data and fitted circles.*

```

1 function draw_circle(x, y, r_, cx_, cy_, r, cx, cy)
2
3     plot(x, y, 'bx');
4     hold on; axis equal; grid on
5
6     theta = linspace(0, 2 * pi, 400);
7     plot(cx_ + r_*cos(theta), cy_ + r_*sin(theta), ...
8         'b-', 'LineWidth', 1.2);
9
10    theta = linspace(0, 2 * pi, 400);
11    plot(cx + r_*cos(theta), cy + r_*sin(theta), ...
12        'r-', 'LineWidth', 1.2);
13
14    xlabel('x'), ylabel('y')
15    title('Noisy circle data: r = 10, (c_x, c_y) = (0, 0)');
16
17    hold off
18 end

```

2. [Subfigure 2.2b](#) displays the objective  $f_{\text{ex1}}(r)$  for  $r \in [9.5, 10.5]$ . The function is quadratic and therefore convex in  $r$ , which can also be shown analytically. The minimum is slightly shifted from  $r = 10$ , reflecting the influence of the measurement noise.
3. [Subfigure 2.2c](#) presents a contour plot of  $f_{\text{ex2}}(r, c_x)$  for  $r \in [0, 30]$  and  $c_x \in [0, 30]$ . The contour lines reveal that  $f_{\text{ex2}}$  is not convex (if it were, all level sets would be convex as well). This nonconvexity can also be established formally. For  $f_{\text{ex3}}$ , visualization would require plotting two-dimensional manifolds in  $\mathbb{R}^3$ , which is considerably more challenging.

### §3 Derivatives of the Objective Function

In order to formulate numerical methods such as Newton–Raphson, Gauss–Newton, or Levenberg–Marquardt, we require the first and second derivatives of the objective function  $f$ . The goal is to express the partial derivatives of  $f$  in terms of the derivatives of the residuals  $F_1, \dots, F_m$ , so that the resulting expressions can be easily adapted to specific least-squares problems.

We use  $i$  and  $j$  as row and column indices, respectively, and denote the summation index by  $\ell$ . Let

$$f(x) = \frac{1}{2} \sum_{\ell=1}^m F_\ell(x)^2, \quad x \in \mathbb{R}^n.$$

For the partial derivatives of  $f$ , the chain rule gives

$$\frac{\partial f}{\partial x_j}(x) = \frac{1}{2} \sum_{\ell=1}^m 2 F_\ell(x) \frac{\partial F_\ell}{\partial x_j}(x) = \sum_{\ell=1}^m F_\ell(x) \frac{\partial F_\ell}{\partial x_j}(x). \quad (2.8)$$

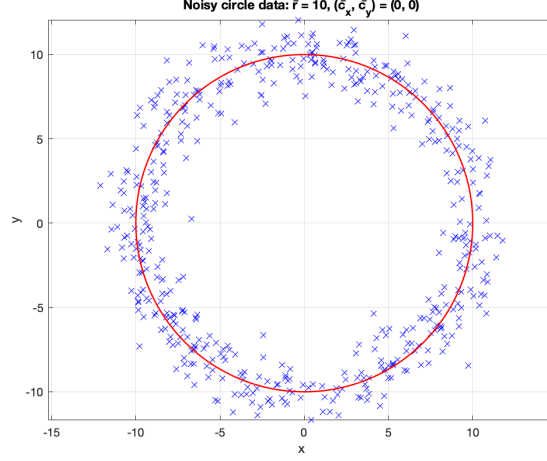
Hence, the gradient is

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(x) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x) \end{pmatrix} = \sum_{\ell=1}^m F_\ell(x) \nabla F_\ell(x) \quad \text{with} \quad \nabla F_\ell(x) := \begin{pmatrix} \frac{\partial F_\ell}{\partial x_1}(x) \\ \vdots \\ \frac{\partial F_\ell}{\partial x_n}(x) \end{pmatrix}. \quad (2.9)$$

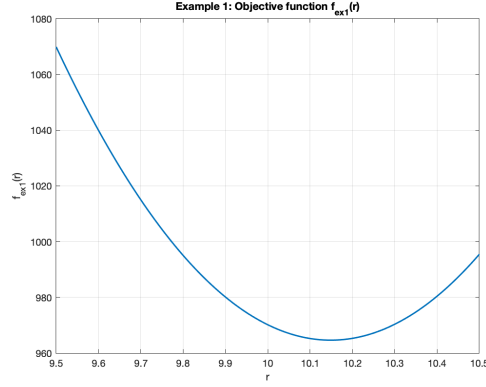
For the second partial derivatives, applying the product rule yields

$$\frac{\partial^2 f}{\partial x_i \partial x_j}(x) = \sum_{\ell=1}^m \left( \frac{\partial F_\ell}{\partial x_i}(x) \frac{\partial F_\ell}{\partial x_j}(x) + F_\ell(x) \frac{\partial^2 F_\ell}{\partial x_i \partial x_j}(x) \right). \quad (2.10)$$

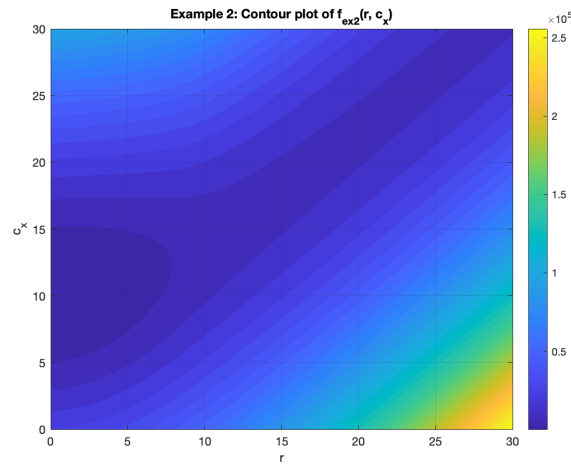
This is the entry of the Hessian matrix  $H_f(x)$  in row  $i$  and column  $j$ . To write this more compactly, it is helpful to recognize how the collection of all products  $\frac{\partial F_\ell}{\partial x_i}(x) \frac{\partial F_\ell}{\partial x_j}(x)$  can



(a) Generated data using  $[x, y] = \text{create\_circ\_data}(500, 10, 0, 0, 1)$  and visualized with  $\text{draw\_circle}(x, y, 10, 0, 0, 10, 0, 0)$ . The noisy points (black) are distributed around the true circle (blue).



(b) Plot of  $f_{\text{ex1}}(r)$  for  $r \in [9.5, 10.5]$ . The objective is convex in  $r$ , though the minimum is slightly shifted from the true radius  $r = 10$  due to noise.



(c) Contour plot of  $f_{\text{ex2}}(r, c_x)$  for  $r \in [0, 30]$  and  $c_x \in [0, 30]$ . The objective is non-convex, showing multiple local minima.

**Figure 2.2:** Illustrations of the test problems. (a) Generated circle data with noise, (b) convex objective  $f_{\text{ex1}}(r)$ , and (c) non-convex objective  $f_{\text{ex2}}(r, c_x)$ .



be represented as a single matrix expression. Consider

$$\begin{aligned}
(\nabla F_1 \quad \dots \quad \nabla F_m) \begin{pmatrix} \nabla F_1^\top \\ \vdots \\ \nabla F_m^\top \end{pmatrix} &= \sum_{\ell=1}^m \nabla F_\ell \nabla F_\ell^\top = \sum_{\ell=1}^m \begin{pmatrix} \frac{\partial F_\ell}{\partial x_1}(x) \\ \vdots \\ \frac{\partial F_\ell}{\partial x_n}(x) \end{pmatrix} \begin{pmatrix} \frac{\partial F_\ell}{\partial x_1}(x) & \dots & \frac{\partial F_\ell}{\partial x_n}(x) \end{pmatrix} \\
&= \sum_{\ell=1}^m \begin{pmatrix} \frac{\partial F_\ell}{\partial x_1}(x) \frac{\partial F_\ell}{\partial x_1}(x) & \dots & \frac{\partial F_\ell}{\partial x_1}(x) \frac{\partial F_\ell}{\partial x_n}(x) \\ \vdots & & \vdots \\ \frac{\partial F_\ell}{\partial x_n}(x) \frac{\partial F_\ell}{\partial x_1}(x) & \dots & \frac{\partial F_\ell}{\partial x_n}(x) \frac{\partial F_\ell}{\partial x_n}(x) \end{pmatrix} \\
&= \begin{pmatrix} \vdots & & \vdots \\ \dots & \sum_{\ell=1}^m \frac{\partial F_\ell}{\partial x_i}(x) \frac{\partial F_\ell}{\partial x_j}(x) & \dots \\ \vdots & & \vdots \end{pmatrix}_{1 \leq i, j \leq n}.
\end{aligned}$$

Thus, the  $(i, j)$ -entry of this matrix is precisely  $\sum_{\ell=1}^m \frac{\partial F_\ell}{\partial x_i}(x) \frac{\partial F_\ell}{\partial x_j}(x)$ , which corresponds exactly to the first term in (2.10).

The Hessian of a single residual function  $F_\ell$  is

$$H_{F_\ell}(x) = \left( \frac{\partial^2 F_\ell}{\partial x_i \partial x_j}(x) \right)_{1 \leq i, j \leq n}.$$

Combining these results with (2.10), we obtain the compact form

$$H_f(x) = \sum_{\ell=1}^m \left( \nabla F_\ell(x) \nabla F_\ell(x)^T + F_\ell(x) H_{F_\ell}(x) \right). \quad (2.11)$$

**Jacobian notation (for later reference).** Let

$$F: \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad F(x) = \begin{pmatrix} F_1(x) \\ \vdots \\ F_m(x) \end{pmatrix}.$$

For the least-squares problem introduced in Definition 2.1.1, the mapping  $F$  is assumed to be twice continuously differentiable. Its derivative  $F'(x)$  is the *Jacobian matrix* (or Jacobian)

$$F'(x) = J(x) = \begin{pmatrix} \frac{\partial F_1}{\partial x_1}(x) & \dots & \frac{\partial F_1}{\partial x_n}(x) \\ \vdots & & \vdots \\ \frac{\partial F_m}{\partial x_1}(x) & \dots & \frac{\partial F_m}{\partial x_n}(x) \end{pmatrix} = \begin{pmatrix} \nabla F_1(x)^T \\ \vdots \\ \nabla F_m(x)^T \end{pmatrix} \in \mathbb{R}^{m \times n}. \quad (2.12)$$

With this notation, the results obtained above can be expressed compactly in matrix form. From (2.9), the gradient of  $f$  can be written as

$$\nabla f(x) = J(x)^T F(x), \quad (2.13)$$

and from (2.11), the Hessian follows as

$$H_f(x) = J(x)^T J(x) + \sum_{\ell=1}^m F_\ell(x) H_{F_\ell}(x). \quad (2.14)$$

The first term  $J(x)^T J(x)$  corresponds to  $\sum_{\ell=1}^m \nabla F_\ell(x) \nabla F_\ell(x)^T$  and is often referred to as the *Gauss–Newton term*. The second term collects all contributions involving the Hessians  $H_{F_\ell}(x)$  of the residuals  $F_\ell$ . It vanishes when the residuals are small, which will motivate the *Gauss–Newton approximation*.

## §4 Newton–Raphson Method

The idea is to determine stationary points  $\hat{x}$ , that is, points for which

$$\nabla f(\hat{x}) = \mathbf{0}.$$

As already mentioned, one may then examine whether such points correspond to local minima.

Define

$$g : \mathbb{R}^n \rightarrow \mathbb{R}^n, \quad g(x) = \nabla f(x).$$

Finding stationary points is therefore equivalent to solving a nonlinear system of equations, a problem known from the course *Numerical Analysis*. Since  $f$  is twice continuously differentiable,  $g$  is continuously differentiable as well. Hence, the Newton method is a natural candidate for computing such points:

$$x_{k+1} = x_k - g'(x_k)^{-1}g(x_k), \quad \text{starting from some } x_0.$$

For the method to be applicable,  $g'(x_k)$  must be invertible at every step. Convergence can only be guaranteed under additional assumptions. Under certain conditions, the convergence is even quadratic if the initial guess is sufficiently close to the solution.

For reference, the following theorem was proven in the lecture.

**Theorem 2.4.1** (Quadratic convergence of the Newton method for systems)

Let  $M \subseteq \mathbb{R}^n$  be open and convex,  $\|\cdot\|$  a norm in  $\mathbb{R}^n$ ,  $\|\cdot\|$  a compatible matrix norm in  $\mathbb{R}^{n \times n}$ , and  $g : M \rightarrow \mathbb{R}^n$  a continuously differentiable function whose Jacobian  $g'(x)$  is invertible for all  $x \in M$ , such that

$$\|(g'(x))^{-1}\| \leq \beta \quad \forall x \in M. \quad (2.15)$$

Furthermore, assume that  $g'(x)$  is Lipschitz continuous on  $M$  with constant  $\gamma$ , i.e.

$$\|g'(x) - g'(y)\| \leq \gamma \|x - y\| \quad \forall x, y \in M. \quad (2.16)$$

Suppose there exists a solution  $\hat{x} \in M$  of  $g(x) = \mathbf{0}$ . The initial guess  $x_0$  satisfies

$$x_0 \in U_\omega := \{x \in \mathbb{R}^n : \|x - \hat{x}\| < \omega\},$$

where  $\omega$  is sufficiently small such that  $U_\omega \subseteq M$ , and there exists a constant  $\alpha$  satisfying

$$\frac{\beta\gamma\omega}{2} \leq \alpha < 1. \quad (2.17)$$

Then the Newton method is well defined and converges quadratically to  $\hat{x}$ .

The assumptions of the theorem are often difficult to verify in practice, and the behavior of the Newton method strongly depends on the specific problem at hand. To illustrate its application and to gain some intuition, we now consider the test problems introduced in Section §2.

**Example 2.4.2** (Application to  $f_{\text{ex1}}$ )

We first consider the simplest case,

$$f_{\text{ex1}}(r) := f(r, 0, 0),$$

that is, we assume the circle is centered at the origin and only the radius  $r$  is unknown. Hence, the center coordinates  $c_x$  and  $c_y$  are fixed, and all partial derivatives of  $F_i$  with respect to  $c_x$  and  $c_y$  vanish. The residuals simplify to

$$F_i(r) = \sqrt{x_i^2 + y_i^2} - r,$$

with

$$\frac{dF_i}{dr} = -1, \quad \frac{d^2 F_i}{dr^2} = 0.$$

Therefore, the gradient and Hessian of  $f_{\text{ex1}}$  are

$$\begin{aligned} g(r) &= \nabla f_{\text{ex1}}(r) = \sum_{i=1}^m F_i(r) \frac{dF_i}{dr} = - \sum_{i=1}^m F_i(r) = - \sum_{i=1}^m \left( \sqrt{x_i^2 + y_i^2} - r \right) \\ &= m r - \sum_{i=1}^m \sqrt{x_i^2 + y_i^2}, \end{aligned}$$

and

$$g'(r) = H_{f_{\text{ex1}}}(r) = \sum_{i=1}^m \left[ \left( \frac{dF_i}{dr} \right)^2 + F_i(r) \frac{d^2 F_i}{dr^2} \right] = \sum_{i=1}^m 1 = m,$$

where  $m$  denotes the number of data points. Consequently, the Newton step becomes particularly simple:

$$r_{k+1} = r_k - \frac{g(r_k)}{g'(r_k)} = r_k + \frac{1}{m} \sum_{i=1}^m F_i(r_k) = \frac{1}{m} \sum_{i=1}^m \sqrt{x_i^2 + y_i^2}.$$

Thus, in a single iteration the method already yields the exact solution, namely the average distance of the data points from the origin.  $\square$

**Example 2.4.3** (Application to  $f_{\text{ex2}}$ ) We now consider the case

$$f_{\text{ex2}}(r, c_x) := f(r, c_x, 0),$$

where the circle center lies on the  $x$ -axis. The  $y$ -coordinate  $c_y$  is fixed to zero, while  $r$  and  $c_x$  are unknown. The residuals are

$$F_i(r, c_x) = \sqrt{(x_i - c_x)^2 + y_i^2} - r.$$

The partial derivatives of  $F_i$  are

$$\frac{\partial F_i}{\partial r} = -1, \quad \frac{\partial F_i}{\partial c_x} = \frac{c_x - x_i}{\sqrt{(x_i - c_x)^2 + y_i^2}},$$

Hence,  $F_i$  is continuously differentiable everywhere except at the isolated points where  $y_i = 0$  and  $c_x = x_i$ , where the denominator vanishes. Consequently,  $f_{\text{ex2}}$  is continuous on  $\mathbb{R}^2$  but not differentiable at these points.

Away from these points,  $F_i$  is twice continuously differentiable, and the only nonzero second partial derivative is

$$\frac{\partial^2 F_i}{\partial c_x^2} = \frac{y_i^2}{\left((x_i - c_x)^2 + y_i^2\right)^{3/2}}.$$

With  $x = (r, c_x)^T$ , the gradient and Hessian of  $f_{\text{ex2}}$  follow from

$$g(x) = \nabla f_{\text{ex2}}(x) = \sum_{i=1}^m F_i(x) \nabla F_i(x), \quad g'(x) = \sum_{i=1}^m \left( \nabla F_i(x) \nabla F_i(x)^T + F_i(x) H_{F_i}(x) \right).$$

Explicitly,

$$g(x) = \sum_{i=1}^m \left( F_i(x) \frac{\begin{pmatrix} -F_i(x) \\ c_x - x_i \end{pmatrix}}{\sqrt{(x_i - c_x)^2 + y_i^2}} \right),$$

and

$$g'(x) = \sum_{i=1}^m \begin{pmatrix} 1 & -\frac{c_x - x_i}{\sqrt{(x_i - c_x)^2 + y_i^2}} \\ -\frac{c_x - x_i}{\sqrt{(x_i - c_x)^2 + y_i^2}} & \frac{(c_x - x_i)^2}{(x_i - c_x)^2 + y_i^2} + F_i(x) \frac{y_i^2}{\left((x_i - c_x)^2 + y_i^2\right)^{3/2}} \end{pmatrix}.$$

In this two-dimensional setting, the Newton update

$$x_{k+1} = x_k - g'(x_k)^{-1} g(x_k)$$

can be computed iteratively. For noise-free data and a starting value close to the solution, the method converges quadratically to the true parameters  $(r, c_x)$ .  $\square$

#### **Example 2.4.4** (Application to $f_{\text{ex3}}$ )

We now consider the general case

$$f_{\text{ex3}}(r, c_x, c_y) := f(r, c_x, c_y),$$

where all three parameters are unknown. The residuals are

$$F_i(r, c_x, c_y) = \sqrt{(x_i - c_x)^2 + (y_i - c_y)^2} - r.$$

For brevity, we define

$$d_x := c_x - x_i, \quad d_y := c_y - y_i, \quad \rho := \sqrt{d_x^2 + d_y^2}.$$

Then the first derivatives of  $F_i$  are

$$\frac{\partial F_i}{\partial r} = -1, \quad \frac{\partial F_i}{\partial c_x} = \frac{d_x}{\rho}, \quad \frac{\partial F_i}{\partial c_y} = \frac{d_y}{\rho}.$$

Hence,  $F_i$  is continuously differentiable everywhere except at the isolated points where  $\rho = 0$ , i.e.  $(c_x, c_y) = (x_i, y_i)$ . Consequently,  $f_{\text{ex3}}$  is continuous on  $\mathbb{R}^3$  but not differentiable at these points.

Away from these points,  $F_i$  is twice continuously differentiable, and the only nonzero second partial derivatives are

$$\frac{\partial^2 F_i}{\partial c_x^2} = \frac{d_y^2}{\rho^3}, \quad \frac{\partial^2 F_i}{\partial c_y^2} = \frac{d_x^2}{\rho^3}, \quad \frac{\partial^2 F_i}{\partial c_x \partial c_y} = -\frac{d_x d_y}{\rho^3}.$$

With  $x = (r, c_x, c_y)^T$ , the gradient and Hessian of  $f_{\text{ex3}}$  follow from

$$g(x) = \nabla f_{\text{ex3}}(x) = \sum_{i=1}^m F_i(x) \nabla F_i(x), \quad g'(x) = \sum_{i=1}^m \left( \nabla F_i(x) \nabla F_i(x)^T + F_i(x) H_{F_i}(x) \right).$$

Explicitly,

$$g(x) = \sum_{i=1}^m \begin{pmatrix} -F_i(x) \\ F_i(x) \frac{d_x}{\rho} \\ F_i(x) \frac{d_y}{\rho} \end{pmatrix},$$

and

$$g'(x) = \sum_{i=1}^m \begin{pmatrix} 1 & -\frac{d_x}{\rho} & -\frac{d_y}{\rho} \\ -\frac{d_x}{\rho} & \frac{d_x^2}{\rho^2} + F_i(x) \frac{d_y^2}{\rho^3} & \frac{d_x d_y}{\rho^2} - F_i(x) \frac{d_x d_y}{\rho^3} \\ -\frac{d_y}{\rho} & \frac{d_x d_y}{\rho^2} - F_i(x) \frac{d_x d_y}{\rho^3} & \frac{d_y^2}{\rho^2} + F_i(x) \frac{d_x^2}{\rho^3} \end{pmatrix}.$$

In this three-dimensional setting, the Newton update

$$x_{k+1} = x_k - g'(x_k)^{-1} g(x_k)$$

can be computed iteratively. For noise-free data and a starting value close to the solution, the method converges quadratically to the true parameters  $(r, c_x, c_y)$ .  $\square$

## §5 Gauss–Newton

In the previous section (Newton–Raphson method), we have seen that even for comparatively simple examples the computation of  $g'(x) = H_f(x)$  can become quite involved. The idea of the Gauss–Newton method is to exploit the specific structure of nonlinear least squares problems and to simplify the expression for the Hessian.

Recall that for the minimization problem

$$f(x) = \frac{1}{2} \sum_{i=1}^m F_i(x)^2,$$

the gradient and Hessian are given by

$$g(x) = J(x)^T F(x), \quad g'(x) = J(x)^T J(x) + \sum_{\ell=1}^m F_\ell(x) H_{F_\ell}(x),$$

where  $J(x)$  denotes the Jacobian matrix of  $F(x)$ ,

$$J(x) = \begin{pmatrix} \nabla F_1(x)^T \\ \vdots \\ \nabla F_m(x)^T \end{pmatrix}.$$

At the exact solution  $\hat{x}$ , the residuals vanish for noise-free data, that is,

$$F_i(\hat{x}) = 0, \quad i = 1, \dots, m.$$

If the data are only slightly perturbed, we may still assume that  $F_i(x) \approx 0$  for  $x$  sufficiently close to  $\hat{x}$ . Under this assumption, the second term in the Hessian expression becomes small, and we obtain the approximation

$$g'(x) = J(x)^T J(x) + \sum_{\ell=1}^m F_\ell(x) H_{F_\ell}(x) \approx J(x)^T J(x).$$

This motivates the *Gauss-Newton method*, in which the full Hessian  $H_f(x)$  is replaced by the simpler approximation  $J(x)^T J(x)$ . The resulting linear system for the Newton step is

$$J(x_k)^T J(x_k) s_k = -J(x_k)^T F(x_k),$$

and the iterate is updated according to

$$x_{k+1} = x_k + s_k.$$

The complete algorithm is summarized in [Figure 2.3](#), which also shows the corresponding MATLAB implementation used in the examples below.

Although we postpone a detailed convergence analysis, it is instructive to note that the Gauss-Newton iteration can be viewed as a fixed point iteration

$$x_{k+1} = \Phi(x_k),$$

where

$$\Phi(x) = x - \left( J(x)^T J(x) \right)^{-1} J(x)^T F(x).$$

For this iteration to be well-defined, it is necessary that

$$\text{rank}(J(x)) = n,$$

so that the matrix  $J(x)^T J(x)$  is invertible.

Local convergence can then, in principle, be established using Banach's fixed point theorem, provided that in a neighbourhood of the solution  $\hat{x}$  the mapping  $\Phi$  is self-mapping and

$$\|\Phi'(\hat{x})\| < 1,$$

where  $\|\cdot\|$  denotes any matrix norm on  $\mathbb{R}^n$ .

**Figure 2.3:** Gauss–Newton method and corresponding MATLAB implementation.

---

**Algorithm 1:** Gauss–Newton algorithm

---

**Data:** Initial guess  $x_0$ , maximum iteration count  $n_{\max}$

**Result:** Approximate solution  $x_k$

```

1 for  $k = 0, 1, 2, \dots, n_{\max}$  do
2   Compute residuals  $b = F(x_k)$  and Jacobian  $A = J(x_k)$ 
3   Solve  $(A^\top A)s_k = -A^\top b$ 
4   Update  $x_{k+1} = x_k + s_k$ 
5   if  $\|b\| \leq 10^{-9}$  then
6     break

```

---

```

1 function [x, k] = gauss_newton(x, F, F_, n_max)
2   arguments
3     x double
4     F (1,1) function_handle
5     F_ (1,1) function_handle
6     n_max (1,1) double {mustBeInteger, mustBeNonnegative}
7   end
8
9   for k = 1:n_max
10    b = F(x);
11    A = F_(x);
12    s = (A'*A) \ (-A'*b);
13    x = x + s;
14    if norm(b) <= 1e-9
15      break;
16    end
17  end
18 end

```

---

## §6 Outlook: Levenberg–Marquardt method

Similar to the Gauss–Newton approach, the idea of the *Levenberg–Marquardt method* is to simplify the Hessian  $H_f(x)$ , yet in a way that yields a more robust approximation than Gauss–Newton does. The method replaces the Hessian by a regularized form

$$g'(x) \approx J(x)^T J(x) + \lambda I, \quad \lambda \geq 0,$$

where  $\lambda$  is a damping (or regularization) parameter and  $I$  denotes the identity matrix.

In the next chapter, which directly follows, we will introduce and analyse the *gradient–descent method* and *trust region methods*. Both of these concepts play an important role here: the gradient–descent method represents the limiting case of Levenberg–Marquardt for large  $\lambda$ , while trust region methods provide a systematic framework for choosing an appropriate damping parameter  $\lambda$ .

The parameter  $\lambda$  thus controls the transition between the Gauss–Newton and a gradient–descent–like behaviour: for  $\lambda = 0$  the method coincides with Gauss–Newton, while for large  $\lambda$  the step direction approaches that of steepest descent.

Choosing a “good” value for  $\lambda$  and understanding why this modification can lead to an improved approximation will be discussed later in the context of trust region methods. Nevertheless, the Levenberg–Marquardt method is already introduced and used in the exercises in order to gain numerical experience with this practically important approach.



## Chapter 3 Unconstrained optimization

In this chapter we consider general optimization problems of the form

$$f(\hat{x}) = \min_{x \in \mathbb{R}^n} f(x),$$

where the objective function  $f : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$  is defined on an open domain  $\Omega$ .

Such problems are called *unconstrained*, because all points  $x \in \mathbb{R}^n$  are admissible; there are no explicit equality or inequality constraints restricting the feasible set.

As the following examples illustrate, least squares problems are special cases of unconstrained optimization problems, since the objective function possesses an additional structural form. Consequently, methods, optimality conditions, and convergence results that are derived in this chapter can be directly applied to least squares problems.

### Example 3.0.1

- (a) For  $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$  twice continuously differentiable and

$$f : \mathbb{R}^n \rightarrow \mathbb{R}, \quad f(x) = \frac{1}{2} \|F(x)\|_2^2,$$

we recover the least squares problem of [Definition 2.1.1](#).

- (b) The *Rosenbrock function*

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}, \quad f(x) = f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

defines a nonlinear optimization problem with a unique global minimum at  $\hat{x} = (1, 1)^T$ , where  $f(\hat{x}) = 0$ . It is often used as a benchmark to test numerical optimization methods, since it exhibits steep gradients in one direction and very flat ones in another. Moreover, these directions are not aligned with the coordinate axes, resulting in a characteristic “banana-shaped” valley (see [Figure 3.1](#)).

- (c) The *Himmelblau function*

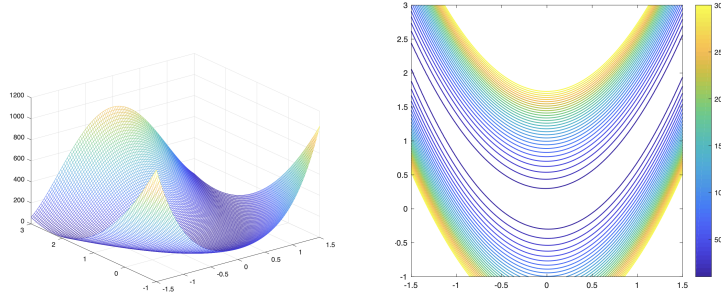
$$f : \mathbb{R}^2 \rightarrow \mathbb{R}, \quad f(x_1, x_2) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2,$$

is a polynomial of degree four and serves as another popular benchmark problem for testing optimization algorithms. It possesses four global minima, each with a function value of zero, approximately at

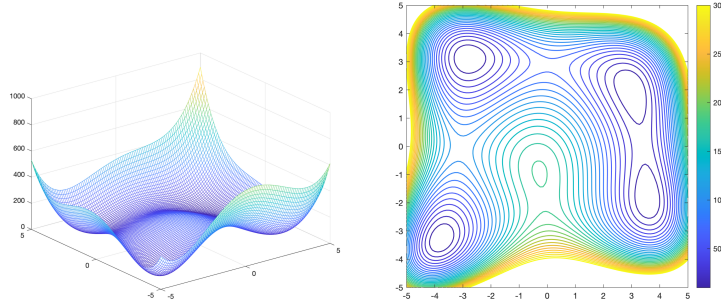
$$(3.0, 2.0), \quad (-2.805, -3.131), \quad (-3.779, -3.283), \quad (3.584, -1.848).$$

In addition, the function has one local maximum at approximately  $(-0.270, -0.924)$  and two saddle points near  $(-0.270, -0.924)$  and  $(-0.550, 1.830)$ , where the gradient vanishes but the Hessian is indefinite. The characteristic shape of the function, with several isolated valleys and a central peak, is illustrated in [Figure 3.2](#).

□



**Figure 3.1:** Contour plot of the Rosenbrock function  $f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$ . The narrow, curved valley illustrates the difficulty of optimization methods based solely on local gradient information.



**Figure 3.2:** Contour plot of the Himmelblau function  $f(x_1, x_2) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2$ . The function has four global minima ( $f = 0$ ), one local maximum, and two saddle points, making it a popular benchmark for nonconvex optimization.

## §1 Optimality conditions and convexity

In this section we recall several necessary and sufficient conditions for local and global extrema. While these results are well known from basic analysis courses, they are summarized here in a form suitable for the context of unconstrained optimization.

### Definition 3.1.1 (Local extrema)

Let  $f : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$  with  $\Omega$  open.

- (a) A point  $\hat{x} \in \Omega$  is called a *local minimum* of  $f$  if there exists a  $\delta > 0$  such that

$$f(x) \geq f(\hat{x}) \quad \forall x \in U_\delta(\hat{x}) := \{x \in \mathbb{R}^n : \|x - \hat{x}\|_2 < \delta\}.$$

It is called a *strict local minimum* if

$$f(x) > f(\hat{x}) \quad \forall x \in \dot{U}_\delta(\hat{x}) := \{x \in \mathbb{R}^n : 0 < \|x - \hat{x}\|_2 < \delta\}.$$

- (b) A point  $\hat{x} \in \Omega$  is called a *local maximum* of  $f$  if there exists a  $\delta > 0$  such that

$$f(x) \leq f(\hat{x}) \quad \forall x \in U_\delta(\hat{x}).$$

It is called a *strict local maximum* if

$$f(x) < f(\hat{x}) \quad \forall x \in \dot{U}_\delta(\hat{x}).$$

□

**Theorem 3.1.2** (First-order necessary optimality condition) Let  $f \in C^1(M; \mathbb{R})$ , where  $M \subseteq \mathbb{R}^n$  is open. If  $f$  attains a local minimum at  $\hat{x} \in M$ , then

$$\nabla f(\hat{x}) = \mathbf{0}.$$

*Proof:* Let  $\delta > 0$  be such that

$$f(x) \geq f(\hat{x}) \quad \forall x \in A := U_\delta(\hat{x}) \subseteq M.$$

For an arbitrary direction  $d \in \mathbb{R}^n \setminus \{\mathbf{0}\}$  with  $\|d\|_2 = 1$ , define the curve

$$h_d : (-\delta, \delta) \rightarrow A, \quad h_d(t) = \hat{x} + td,$$

and the scalar function

$$g_d : (-\delta, \delta) \rightarrow \mathbb{R}, \quad g_d(t) = f(h_d(t)) = f(\hat{x} + td).$$

Then

$$g_d(0) = f(\hat{x}), \quad g_d(t) = f(\hat{x} + td) \geq f(\hat{x}) = g_d(0) \quad \forall t \in (-\delta, \delta).$$

Hence,

$$\lim_{t \rightarrow 0^+} \frac{g_d(t) - g_d(0)}{t} \geq 0, \quad \lim_{t \rightarrow 0^-} \frac{g_d(t) - g_d(0)}{t} \leq 0.$$

Since both one-sided derivatives exist and have opposite signs, it follows that

$$g'_d(0) = 0. \tag{*}$$

By the chain rule,

$$g'_d(t) = f'(h_d(t)) h'_d(t) = \nabla f(h_d(t))^T d.$$

From (\*) it follows that

$$\nabla f(\hat{x})^T d = 0 \quad \text{for all directions } d \in \mathbb{R}^n \text{ with } \|d\|_2 = 1.$$

Choosing in particular the canonical unit vectors  $e_i$  for  $i = 1, \dots, n$ , we obtain

$$0 = \nabla f(\hat{x})^T e_i = \frac{\partial f}{\partial x_i}(\hat{x}), \quad i = 1, \dots, n.$$

Hence, all partial derivatives vanish at  $\hat{x}$ , and therefore

$$\nabla f(\hat{x}) = \mathbf{0}.$$

□

The necessary (but not sufficient) condition for local extrema motivates the following definition.

**Definition 3.1.3** (Stationary points and saddle points)

Let  $f \in C^1(M; \mathbb{R})$ , where  $M \subseteq \mathbb{R}^n$  is open.

(a) A point  $\hat{x} \in M$  is called a *stationary point* if

$$\nabla f(\hat{x}) = \mathbf{0}.$$

- (b) A stationary point  $\hat{x} \in M$  is called a *saddle point* if it is neither a local minimum nor a local maximum, that is, if for every  $\delta > 0$  there exist points  $a, b \in U_\delta(\hat{x}) \cap M$  such that

$$f(a) < f(\hat{x}) \quad \text{and} \quad f(b) > f(\hat{x}).$$

□

**Remark 3.1.4** (Stability of positive definiteness of the Hessian)

For the forthcoming sufficient optimality condition we will assume that  $f \in C^2(\Omega)$  on an open set  $\Omega \subseteq \mathbb{R}^n$ .

If the Hessian  $H_f(\hat{x})$  is positive definite at  $\hat{x} \in \Omega$ , then  $H_f(x)$  is also positive definite on some open neighbourhood  $U_\delta(\hat{x}) \subset \Omega$ . In this remark, we write

$A \succ 0$  to denote that the matrix  $A$  is positive definite,

*Proof sketch (via Sylvester's criterion).*

By Sylvester's criterion,  $H_f(\hat{x}) \succ 0$  iff all *leading* principal minors

$$D_k(\hat{x}) := \det((H_f(\hat{x}))_{1:k, 1:k}), \quad k = 1, \dots, n,$$

are strictly positive. Since  $f \in C^2$ , every entry of  $H_f(x)$  is continuous in  $x$ , hence so is each determinant  $D_k(x)$ . Therefore, for every  $k$  there exists  $\delta_k > 0$  such that  $D_k(x) > \frac{1}{2}D_k(\hat{x}) > 0$  for all  $x \in U_{\delta_k}(\hat{x})$ . With  $\delta := \min_k \delta_k$  we obtain  $D_k(x) > 0$  for all  $k$  and all  $x \in U_\delta(\hat{x})$ , hence  $H_f(x) \succ 0$  there.

□

**Theorem 3.1.5** (Second-order sufficient optimality condition)

Let  $f \in C^2(M; \mathbb{R})$ , where  $M \subseteq \mathbb{R}^n$  is open, and let  $\hat{x} \in M$  satisfy

(i)  $\nabla f(\hat{x}) = \mathbf{0}$ ,

(ii)  $H_f(\hat{x})$  is positive definite, that is,

$$d^T H_f(\hat{x}) d > 0 \quad \forall d \in \mathbb{R}^n \setminus \{\mathbf{0}\}.$$

Then  $\hat{x}$  is a *strict local minimum* of  $f$ .

*Proof:* By the continuity of the Hessian  $H_f$ , there exists a  $\delta > 0$  such that  $H_f(x) \succ 0$  for all  $x \in U_\delta(\hat{x})$ .

For an arbitrary but fixed  $x \in U_\delta(\hat{x})$ , define the path

$$h : [0, 1] \rightarrow \mathbb{R}^n, \quad h(t) = \hat{x} + td, \quad \text{where } d := x - \hat{x},$$

and the scalar function

$$g : [0, 1] \rightarrow \mathbb{R}, \quad g(t) = f(h(t)).$$

Then  $g(0) = f(\hat{x})$  and  $g(1) = f(x)$ . By Taylor's theorem, there exists some  $\xi \in (0, 1)$  such that

$$g(1) = g(0) + g'(0) + \frac{1}{2}g''(\xi).$$

By the chain rule,

$$g'(t) = \nabla f(h(t))^T h'(t) = \nabla f(h(t))^T d, \quad \text{hence } g'(0) = \nabla f(\hat{x})^T d = 0$$

by assumption (i). Differentiating again gives

$$g''(t) = d^T H_f(h(t)) d.$$

Since  $H_f(h(t))$  is positiv definit for all  $t \in [0, 1]$ , we have  $g''(t) > 0$  and therefore

$$g(1) - g(0) = \frac{1}{2}g''(\xi) = \frac{1}{2}d^T H_f(h(\xi)) d > 0.$$

Hence  $f(x) = g(1) > g(0) = f(\hat{x})$ . Because  $x \in \dot{U}_\delta(\hat{x})$  was arbitrary,  $\hat{x}$  is a strict local minimum of  $f$ .  $\square$

**Theorem 3.1.6** (Second-order necessary optimality condition (minimum))

Let  $f \in C^2(\Omega)$  on an open set  $\Omega \subseteq \mathbb{R}^n$  and let  $\hat{x} \in \Omega$  be a local minimum of  $f$ . Then

$$\nabla f(\hat{x}) = \mathbf{0} \quad \text{and} \quad H_f(\hat{x}) \succeq 0.$$

*Sketch / exercise:* The first part follows from the first-order necessary condition. To show that the Hessian is positive semidefinite, argue analogously to the proof of the second-order *sufficient* condition, using that  $H_f$  is continuous and that  $f$  attains a local minimum at  $\hat{x}$ .  $\square$

## §2 Direct search methods

## §3 Gradient descent methods

## §4 Trust-Region Methods

## Anhang A   Supplementary Notes to Chapters

### §1   Least Square

# Bibliography

- [1] Horn, Roger A. und Charles R. Johnson: *Matrix Analysis*, 2. Auflage, Cambridge University Press, 2001.